

CNNs: Visión Artificial

Módulo 4 · Deep Learning

Filtros convolucionales, pooling, ResNet, EfficientNet, transfer learning desde ImageNet y flujo de trabajo completo para sistemas de vision artificial en empresa.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

CNNs: Redes Convolucionales para Visión Artificial

El ojo humano procesa imágenes de forma jerárquica: primero detecta bordes y gradientes, luego formas simples, después partes de objetos, y finalmente objetos completos y sus relaciones. Las redes convolucionales (CNNs) fueron diseñadas para replicar esta jerarquía de forma aprendida a partir de datos. Desde 2012, cuando AlexNet revolucionó ImageNet, las CNNs han sido la arquitectura dominante para visión artificial, y siguen siendo la solución correcta para la mayoría de los problemas de imagen en 2025.

En empresa, las aplicaciones de visión artificial con CNNs son extensas: control de calidad en manufactura, diagnóstico médico por imagen, reconocimiento de documentos y formularios, detección de anomalías visuales en infraestructura, y análisis de vídeo en seguridad. La democratización de los modelos preentrenados en Hugging Face y torchvision ha reducido el coste de entrada enormemente: con 500-1.000 imágenes etiquetadas y una tarde de trabajo, es posible construir un sistema de inspección visual que supere a la inspección humana manual en velocidad y consistencia.

Los componentes de una CNN

La capa convolucional: detectar patrones locales

Una capa convolucional aplica un conjunto de filtros (kernels) sobre la imagen de entrada. Cada filtro es una pequeña matriz de pesos (típicamente 3×3 o 5×5) que se desliza por toda la imagen, calculando el producto punto con cada región de la imagen del mismo tamaño. El resultado es un "mapa de características" (feature map) que muestra dónde en la imagen hay activación alta para ese filtro. Un filtro de 3×3 que detecta bordes verticales producirá un feature map con valores altos donde haya bordes verticales.

Las CNNs aprenden automáticamente qué filtros son útiles durante el entrenamiento. Las primeras capas aprenden detectores de bajo nivel (bordes, colores, texturas). Las capas intermedias combinan esos patrones para detectar formas más complejas (esquinas, curvas, partes de objetos). Las últimas capas aprenden detectores de alto nivel (ojos de un perro, rueda de un coche, tumor en una imagen médica). Esta jerarquía aprendida es la clave de la potencia de las CNNs.

Pooling: reducir dimensionalidad preservando información

Después de las capas convolucionales, las capas de pooling reducen la dimensión espacial de los feature maps. Max pooling toma el valor máximo de cada región (captura si el patrón está presente). Average pooling toma la media. Global Average Pooling (GAP) reduce toda la dimensión espacial a un único valor por canal. El pooling hace al modelo invariante a pequeñas traslaciones: "gato en la esquina superior izquierda" vs "gato en el centro" produce activaciones similares después del pooling.

Arquitecturas CNN clásicas y modernas

LeNet (1989): la pionera. 5 capas, diseñada para reconocimiento de dígitos. AlexNet (2012): primera CNN grande en GPU, 60M parámetros. VGG (2014): profundidad con filtros 3×3 uniformes, muy usada como backbone. ResNet (2016): conexiones residuales, permite redes de 50-152+ capas. EfficientNet (2019): escala de forma óptima profundidad, anchura y resolución, muy eficiente. ConvNeXt (2022): CNN moderna que toma ideas de Vision Transformers, compite con ViT en rendimiento manteniendo la eficiencia de las CNNs.

SECCIÓN 3 · CÓMO FUNCIONA

El flujo completo de procesamiento en una CNN

El flujo es: imagen ($H \times W \times 3$) → capas conv+relu repetidas (reducen $H \times W$, aumentan canales C) → pooling (reduce $H \times W$) → más capas conv → Global Average Pooling ($H \times W$ desaparece, queda un vector de C dimensiones) → capas densas (opcionales) → softmax → probabilidades de clase. La profundidad de la red (número de capas conv) y la anchura (número de filtros por capa) son los hiperparámetros principales que controlan la capacidad del modelo.

Stride y padding son dos parámetros de las capas convolucionales. Stride controla el paso del deslizamiento del filtro (stride=2 produce feature maps de la mitad de tamaño). Padding añade ceros alrededor de la imagen (same padding mantiene las dimensiones, valid padding las reduce). Dilation (dilated convolution) añade espacios entre los elementos del filtro, ampliando el campo receptivo sin aumentar el número de parámetros: útil para segmentación semántica.

Batch Normalization después de cada capa convolucional es prácticamente universal en las CNNs modernas: estabiliza el entrenamiento, permite tasas de aprendizaje mayores, y actúa como regularizador implícito. Data augmentation (transformaciones aleatorias de las imágenes durante el entrenamiento: rotaciones, recortes, inversiones, cambios de color) es la técnica más efectiva para reducir el sobreajuste con pocos datos en visión artificial.

SECCIÓN 4 · EJEMPLOS REALES

- Control de calidad en Foxconn (manufactura de electronics): CNNs entrenadas en imágenes de defectos de fabricación (soldaduras defectuosas, arañazos, componentes mal posicionados) con precisión superior al 99.5% y velocidades de inspección de cientos de imágenes por segundo, imposibles para inspectores humanos.
- Detección de retinopatía diabética (Google Health): EfficientNet entrenado sobre 128.000 imágenes de fondo de ojo con etiquetas de oftalmólogos. En el estudio de validación en clínicas de Tailandia, el modelo igualó o superó a especialistas en la detección de retinopatía diabética grave, con el potencial de llevar diagnóstico especializado a áreas sin acceso a oftalmólogos.
- Reconocimiento automático de documentos (Abbyy, Hyperscience): CNNs para OCR y comprensión de layout en formularios, facturas y contratos. Reducen el tiempo de procesamiento de documentos de horas a segundos con tasas de error comparables a la revisión manual.
- Detección de objetos con YOLO (You Only Look Once) en seguridad: arquitectura CNN que detecta y clasifica múltiples objetos en una imagen en tiempo real (~30ms por imagen). Usado en sistemas de seguridad perimetral, conteo de personas, y análisis de tráfico.

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: No usar data augmentation. Con pocos datos de imagen (cientos a pocos miles de ejemplos), el data augmentation puede ser tan importante como la arquitectura. Transformaciones como RandomHorizontalFlip, RandomCrop, ColorJitter y RandomRotation multiplican efectivamente el tamaño del dataset y reducen el sobreajuste significativamente.

Error 2: Usar imágenes sin normalizar. Las CNNs preentrenadas en ImageNet esperan imágenes normalizadas con la media y desviación estándar de ImageNet (mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]). No aplicar esta normalización produce representaciones que son incompatibles con los pesos preentrenados, degradando el rendimiento del transfer learning.

Error 3: No congelar las capas base en transfer learning con pocos datos. Con 500 imágenes de entrenamiento, si se actualizan todos los parámetros de un ResNet-50 preentrenado (25M parámetros), hay sobreajuste severo. Las primeras capas (que detectan bordes y texturas básicas) deben congelarse; solo las últimas capas (más específicas) deben actualizarse.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

El flujo de trabajo estándar para un proyecto de visión artificial con CNN: define el problema (clasificación, detección, segmentación), recopila y etiqueta datos (herramientas: Label Studio, Roboflow), elige un modelo preentrenado (ResNet50 para clasificación, EfficientDet para detección, SegFormer para segmentación), aplica transfer learning con las capas base congeladas, monitoriza el entrenamiento con las métricas correctas (accuracy para clasificación balanceada, mAP para detección), y despliega el modelo en producción.

Para la gestión de imágenes en PyTorch: usa torchvision.transforms para las transformaciones de datos y data augmentation, torchvision.datasets para datasets estándar, torchvision.models para modelos preentrenados. Para datasets propios: ImageFolder de torchvision o un Dataset personalizado subclassificando torch.utils.data.Dataset.

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M4-T1 (Redes profundas): las CNNs son redes profundas especializadas con capas convolucionales en lugar de densas.
- M4-T5 (Transfer learning): casi todos los proyectos de visión artificial en empresa usan transfer learning desde modelos preentrenados en ImageNet.
- M5 (LLMs): los modelos multimodales (GPT-4V, Claude Sonnet, Gemini) integran un encoder CNN o ViT con un decoder de lenguaje para procesar imágenes y texto conjuntamente.

SECCIÓN 8 · RESUMEN EJECUTIVO

Las CNNs aplican filtros convolucionales aprendidos que detectan características jerárquicamente: bordes → formas → partes → objetos. Los componentes clave son: capas convolucionales (filtros 3×3, stride, padding), pooling (reduce dimensionalidad), BatchNorm (estabiliza entrenamiento) y conexiones residuales (ResNet, permite redes

profundas). Las arquitecturas modernas son EfficientNet (equilibrio rendimiento/eficiencia) y ConvNeXt (CNN modernizada con ideas de Transformers). En empresa: transfer learning desde modelos preentrenados en ImageNet con data augmentation agresiva. Con 500-5.000 imágenes etiquetadas por clase se obtienen sistemas de inspección visual production-ready.